Faculté des Sciences et Ingénierie
Département Informatique

🎙️ UVoice

Plateforme de débat universitaire

De l'argumentation à l'aide à la décision

Étudiants :	-SEKHER NADINE -OULD LHADJ NARIMANE -MENASRIA ABDELDJALLIL -DO CECILIA -TUYISHIME CEDRIC
Enseignants:	- Caren Al Anaissy - Jordan Thieyre
Année universitaire :	2025-2026
Unité d'enseignement :	Projet de développement , LU2IN013

« Parce que chaque voix compte. »

Remerciements

Nous tenons à exprimer notre profonde gratitude à **Mme Caren Al Anaissy** pour son enseignement passionnant sur la manière de construire, analyser et structurer un débat de façon rigoureuse.

Nous adressons également nos sincères remerciements à **M. Jordan Thieyre** pour son suivi attentif tout au long des sprints, son regard critique et ses conseils avisés.

Nous remercions chaleureusement notre scrum master **SEKHER NADINE** pour son sérieux, sa rigueur et son investissement sans faille.

Nous remercions également l'ensemble des utilisateurs ayant testé la plateforme et contribué, par leurs retours, à l'amélioration de l'interface et des fonctionnalités.

Sommaire

Introduction **4**
1. Manuel Utilisateur 5
1.1 Installer et lancer l'application (pour l'utilisateur qui héberge) 5
1.2 Première connexion (sans mot de passe) 5
1.3 Page d'accueil et création d'un débat 6
1.4 Menu utilisateur (⋮) 7
1.5 Gestion des thèmes (admin uniquement) 7
1.6 Créer un débat 8
1.7 Participer à un débat - interface de débats 8
1.7.1 Le graphe interactif 9
1.7.2 Panneau latéral (clic sur un argument) 9
1.7.3 Résultat du débat (en haut du graphe) 9
1.8 Mon historique 10
2.1 Vue d'ensemble de l'architecture 11
2.2 Choix techniques 12
2.3 Modèle argumentatif – Besnard & Hunter étendu 12
2.3.1 Rôle des favoris 13
2.3.2 Choix de l'algorithme de calcul 14
2.4 Gestion des données (relations clés) 14
3. Rapport de test 15
3.1 Introduction 15
3.2 Tests unitaires 15
3.3 Tests d'intégration 16
3.4 Tests fonctionnels (bout en bout) 17
3.5 Limites identifiées 17
4. Rapport carbone 18
4.1 Introduction 18
4.2 Protocole de mesure 18
4.3 Résultats 18
4.4 Interprétation 19
4.5 Recommandations 19
4.6 Conclusion 19
Conclusion générale 20
Glossaire 21

Introduction

Dans le cadre du cours *« Online debating platforms: from argument mapping to decision-aiding »* à Sorbonne Université, nous avons développé UVoice, une plateforme de débat universitaire.

L'objectif est de permettre à des étudiants, professeurs ou administrateurs de :

- structurer des débats sous forme d'arbre argumentatif,
- évaluer la pertinence de chaque argument (notes de 1 à 5 étoiles),
- calculer automatiquement la force des arguments à l'aide d'une extension de la fonction de catégorisation de Besnard & Hunter,
- aider à la décision en déterminant quel camp l'emporte.

Ce rapport présente l'ensemble du travail réalisé, de l'utilisation de la plateforme à son architecture technique, en passant par les tests et l'impact environnemental.

1. Manuel Utilisateur

1.1 Installer et lancer l'application (pour l'utilisateur qui héberge)

Si vous utilisez simplement le site en ligne, passez à la section 1.2.

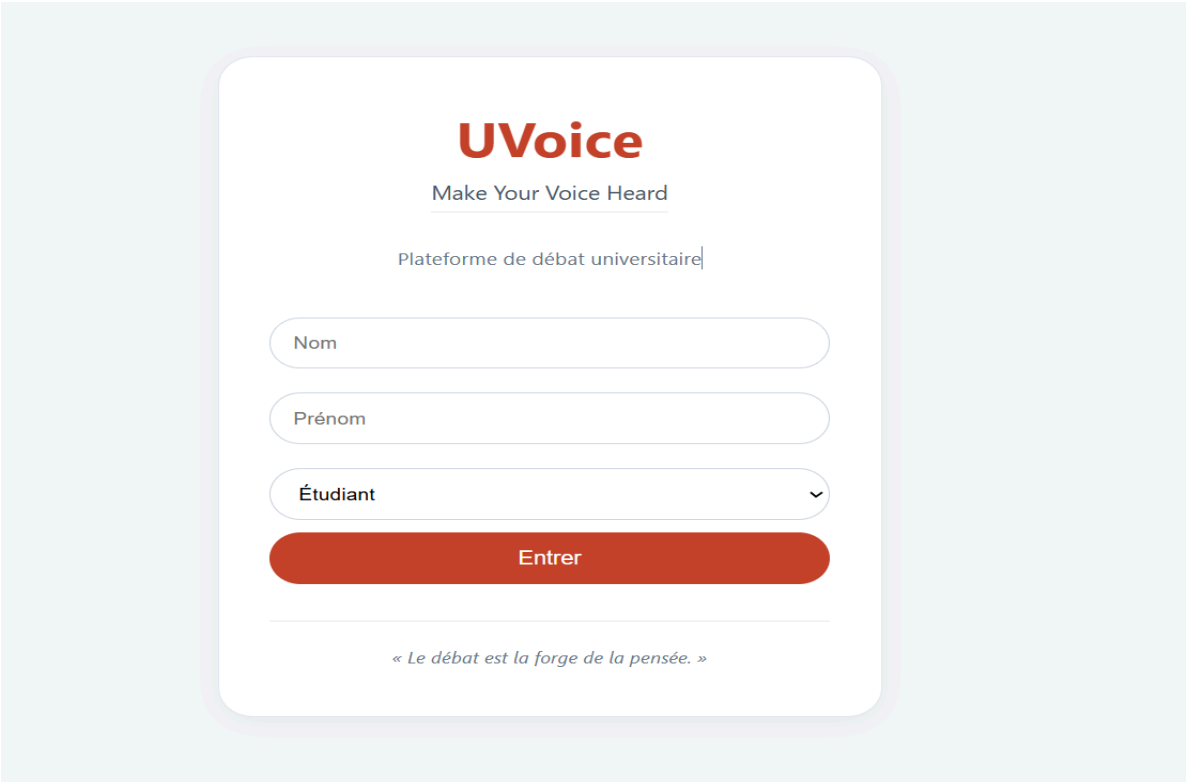
1. Assurez-vous d'avoir **Python 3.10** ou plus récent.
2. Ouvrez un terminal dans le dossier du projet.
3. Lancez la commande :
   - pip install -r requirements.txt (cette commande installe toutes les bibliothèques Python répertoriées dans le fichier requirements.txt, dont Flask pour le serveur et SQLAlchemy pour la base de données).

4. Démarrez le serveur :
   - python routes.py
5. Ouvrez votre navigateur à l'adresse : http://127.0.0.1:5000

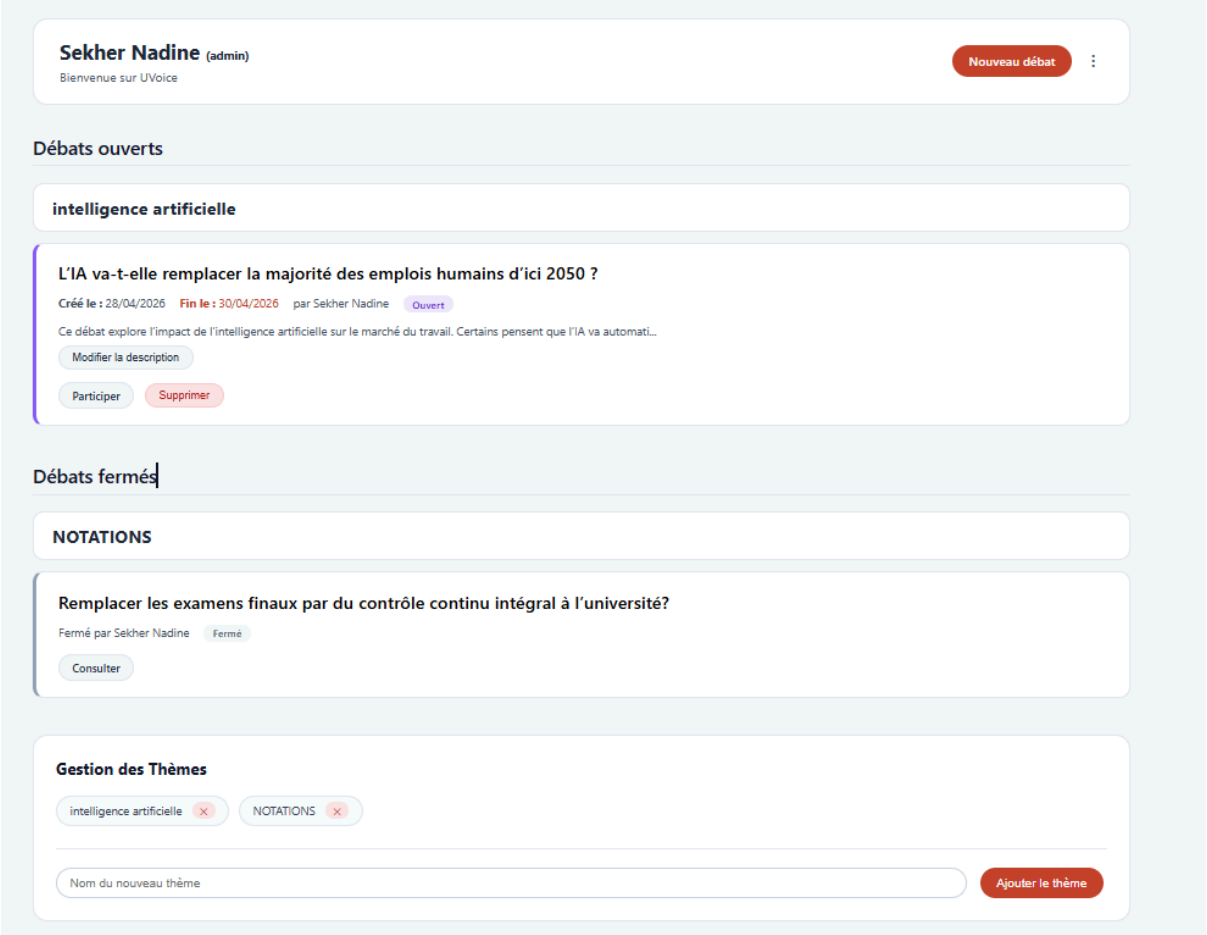
1.2 Première connexion (sans mot de passe)

- Saisissez votre **nom**, **prénom** et votre **rôle** (étudiant, professeur ou administrateur).
- Cliquez sur **Entrer**. (*Votre compte est créé automatiquement.)*

Rôle : les administrateurs peuvent supprimer tout argument/débat et eux seuls peuvent gérer les thèmes.



1.3 Page d’accueil et création d’un débat



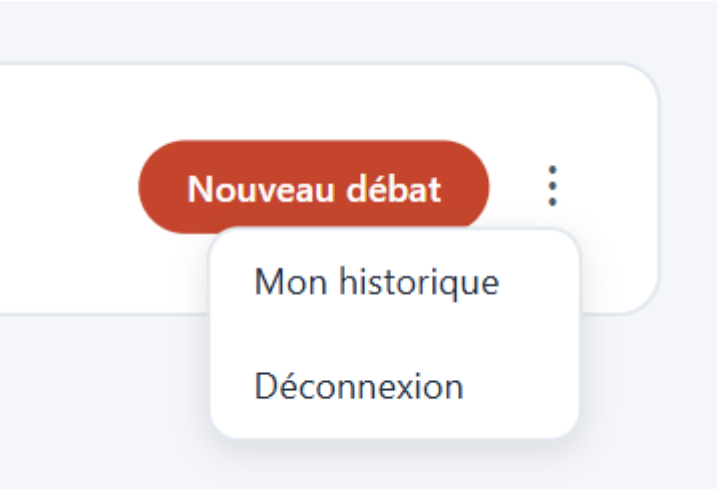
Les débats :

- Les débats sont regroupés par **thème**.
- **Couleur violette** : débat **ouvert** (vous pouvez participer).
- **Couleur grise** : débat **fermé** (consultation seulement).

Actions possibles:

- **Nouveau débat** : créez votre propre sujet.
- **Participer** : entrez dans un débat ouvert.
- **Consulter** : visualisez un débat fermé (plus de participations).
- **Supprimer** : disponible pour l’auteur ou les admins.

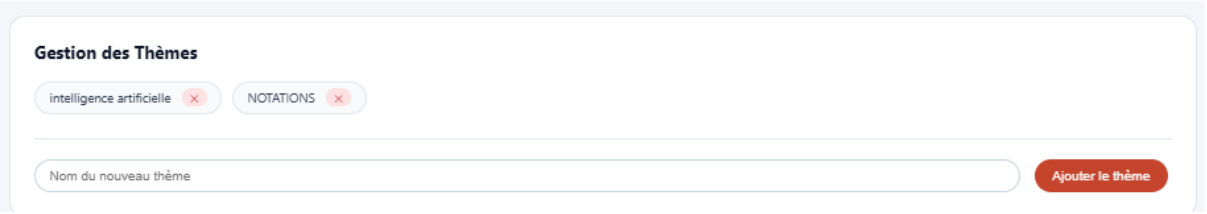
1.4 Menu utilisateur (⋮)



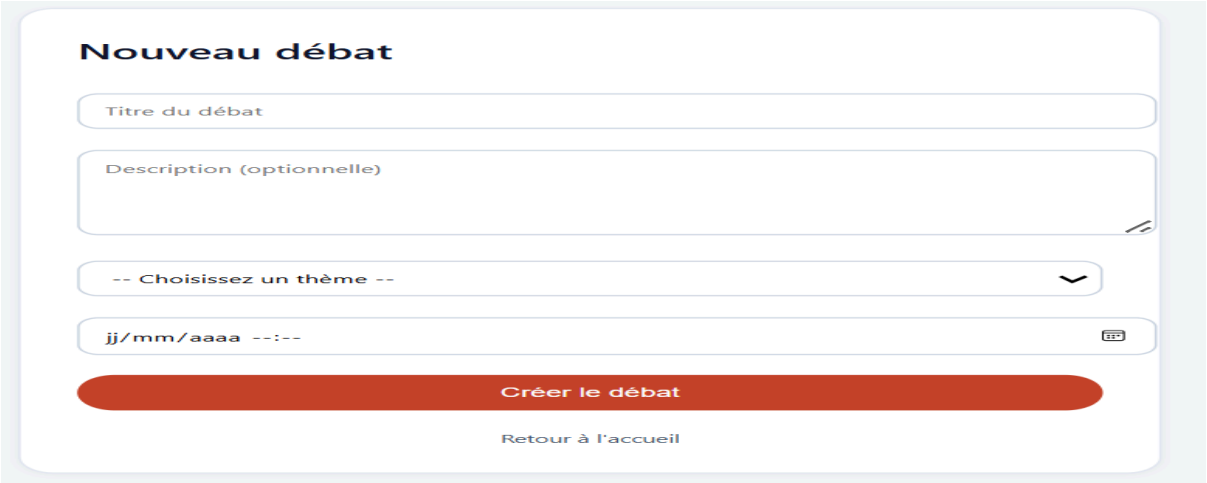
- **Mon historique** : retrouvez toutes vos créations , vos arguments, évaluations et coup de cœur .
- **Déconnexion** : quittez la session.

1.5 Gestion des thèmes (admin uniquement)

- Ajoutez un thème via le formulaire en bas de page.
- Supprimez un thème (croix rouge à côté du nom) – *tous ses débats seront supprimés*.

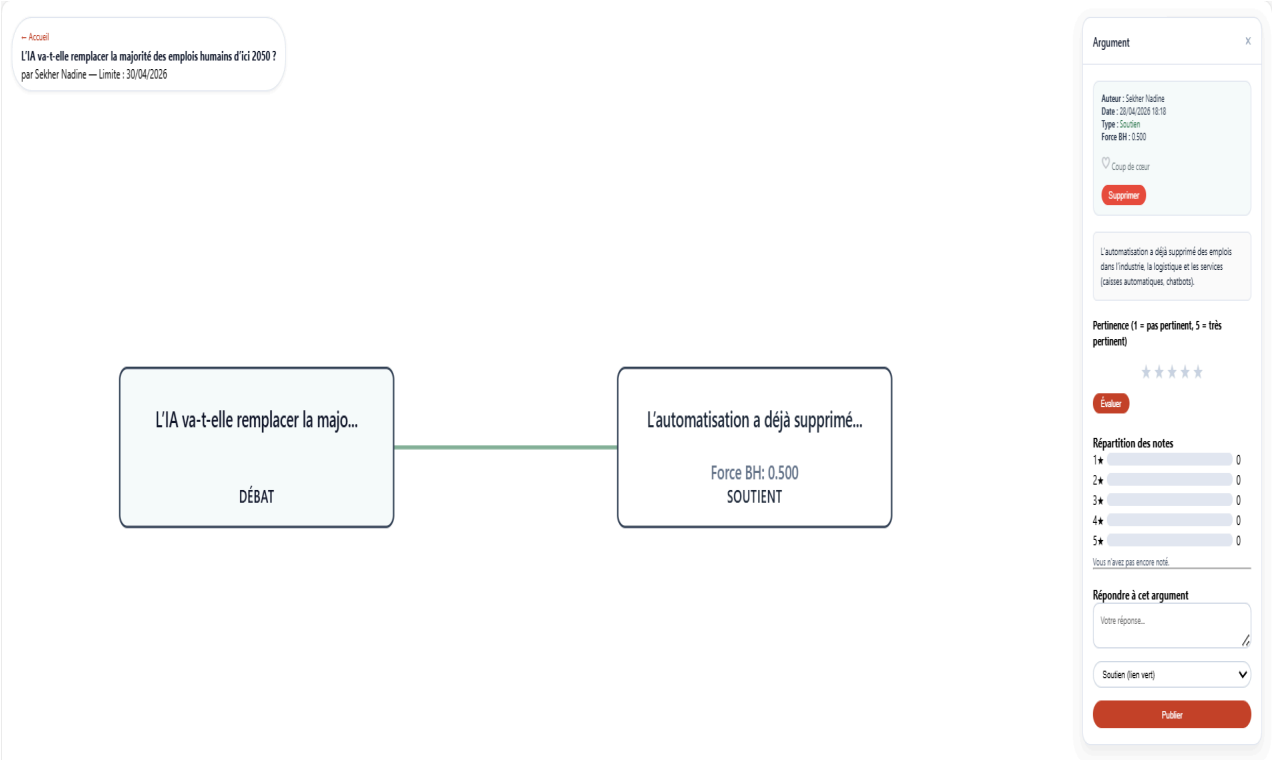


1.6 Créer un débat



1. Cliquez sur **Nouveau débat**.
2. Remplissez les champs :
   a. **Titre du débat** (obligatoire)
   b. **Description** (facultative) modifiable , uniquement si admin ou auteur
   c. **Thème** (obligatoire)
   d. **Date limite** (facultative – après cette date, le débat devient fermé)
3. Validez.
   Le débat apparaît immédiatement dans la liste des débats ouverts.

1.7 Participer à un débat - interface de débats



1.7.1 Le graphe interactif

- **Nœuds** : chaque bulle représente un argument.
- **Liens** : les flèches indiquent la relation.
  - Flèche **verte** = soutien.
  - Flèche **rouge pointillée** = attaque.
- **Zoom / panoramique** : utilisez la molette et le glisser-déposer.

1.7.2 Panneau latéral (clic sur un argument)

Quand vous cliquez sur un argument, un panneau s'ouvre à droite.

a) Informations générales

- Auteur, date, type (soutien/attaque), force BH (Besnard-Hunter).

b) Évaluation de pertinence (1 à 5 étoiles)

- Cliquez sur les étoiles pour noter l'argument.
- **Histogramme** : visualisez la répartition des notes données par tous les utilisateurs.
- Votre propre note est mise en évidence (et vous pouvez la modifier).

c) Favoris /Coup de coeur

- Si un argument vous semble pertinent , cliquez sur le coeur.
  Cela influencera le calcul du résultat du débat.

d) Supprimer (si vous êtes l'auteur ou admin)

- Supprime définitivement l'argument (et toutes ses réponses).

e) Répondre à l'argument

- Choisissez le type : **Soutien** (vert) ou **Attaque** (rouge).
- Rédigez votre réponse, puis **Publier**.
  Le nouvel argument apparaît dans l'arbre.

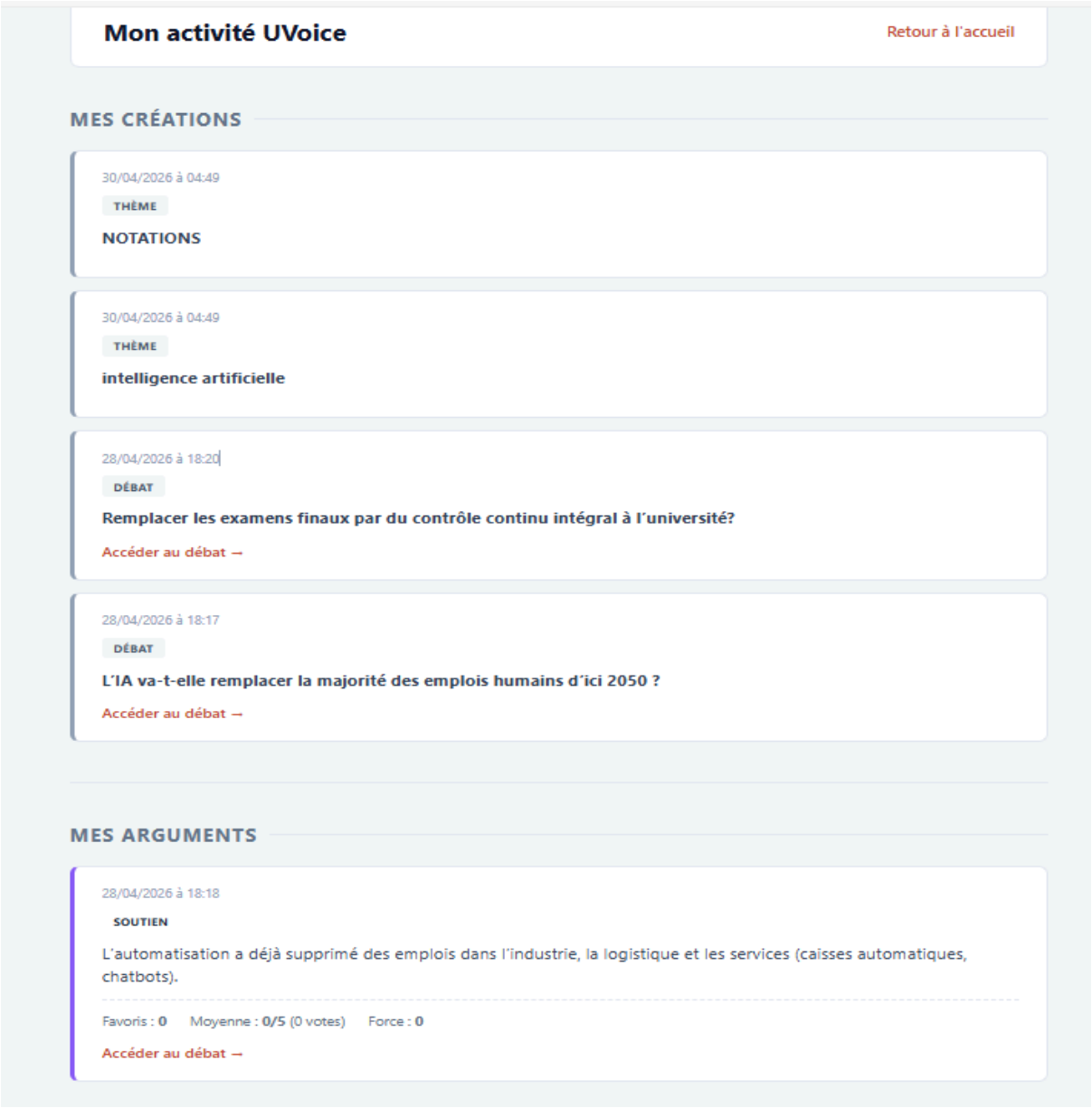
1.7.3 Résultat du débat (en haut du graphe)

- Scores **POUR** et **CONTRE** calculés à partir de la force des arguments et du nombre de favoris.
- Mention du vainqueur.

1.8 Mon historique

Accès via le menu (⁝) → **Mon historique**.

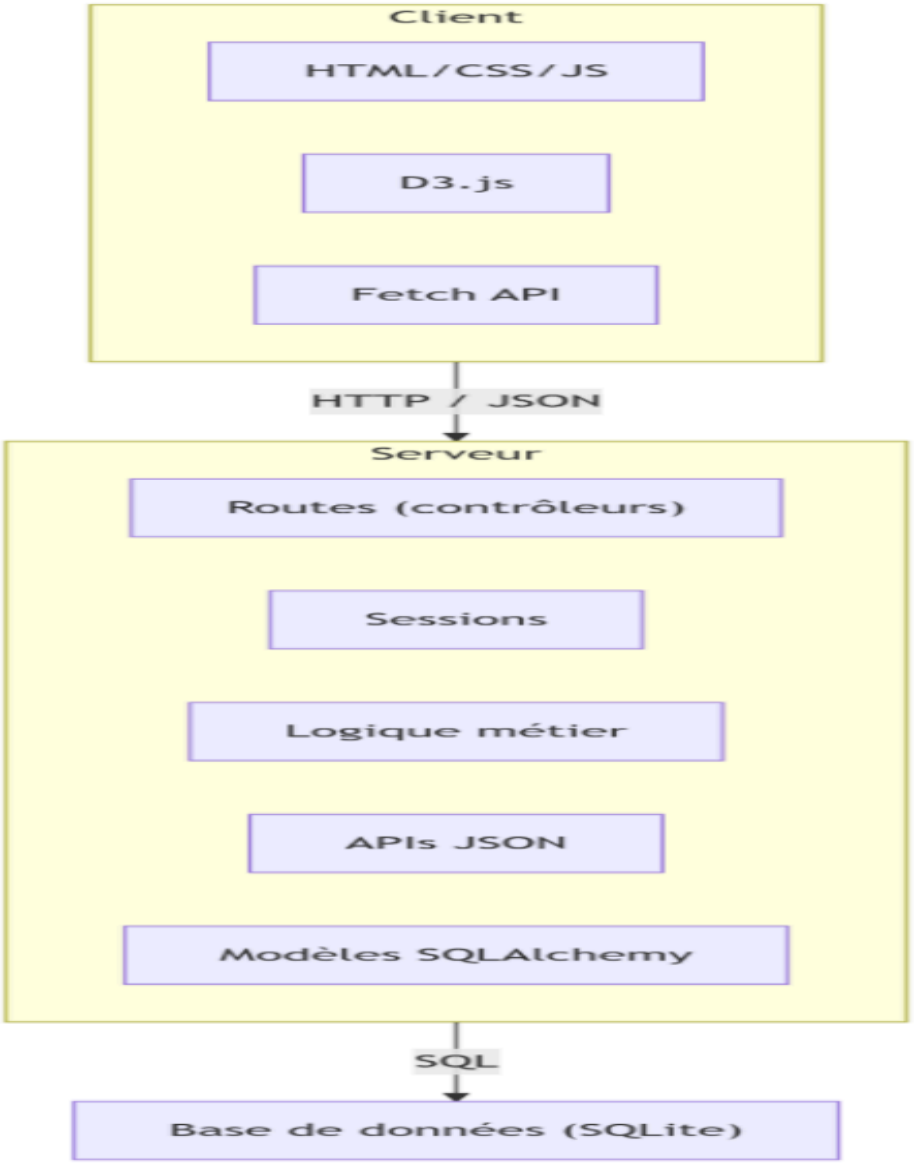
- Vous y trouvez, classés par date :
  - Les arguments que vous avez créés.
  - Les évaluations que vous avez données (avec la note en étoiles).
  - Les arguments favoris(coup de coeur)
  - Les votes (pour/contre) sur les débats.



2-Architecture du projet et choix techniques

2.1 Vue d'ensemble de l'architecture

L'application UVoice suit une **architecture trois tiers** classique, séparant clairement la présentation, la logique métier et les données.



Architecture trois tiers

- **Client** : navigateur web, exécute HTML/CSS/JS, la librairie D3.js pour le graphe, et des appels AJAX (Fetch API) pour les données dynamiques.
- **Serveur** : application Flask qui expose les routes (contrôleurs), gère les sessions utilisateur, implémente la logique métier (calcul des forces, favoris, évaluations) et fournit des APIs JSON.
- **Base de données** : SQLite (fichier `uvoice.db`) interrogée via SQLAlchemy.

2.2 Choix techniques

- **Backend** : Python 3.10 avec Flask (micro-framework léger, pas de structure imposée, sessions intégrées).
- **Base de données** : SQLAlchemy comme ORM, SQLite en développement (fichier unique), PostgreSQL recommandé pour la production (transparent grâce à SQLAlchemy).
- **Frontend** : HTML/CSS/JS pur (pas de framework lourd), styles sobres (vert pour soutien, rouge pour attaque).
- **Visualisation** : D3.js pour le graphe interactif (contrôle total des nœuds, liens, flèches, zoom/pan).
- **Communications** : Fetch API (AJAX) pour charger dynamiquement forces et statistiques sans rechargement.
- **Sessions** : sessions signées Flask (cookie chiffré) pour identifier l'utilisateur (favoris, évaluations, historique).

2.3 Modèle argumentatif – Besnard & Hunter étendu

Le cœur du projet est le calcul de la force de chaque argument. Nous avons implémenté une **version étendue de la fonction de catégorisation de Besnard & Hunter**, qui prend en compte à la fois les **soutiens** et les **attaques**. La formule normalisée retenue est :

$$v(a) = \frac{w(a) + \sum_{s \in Sup(a)} v(s)}{1 + \sum_{s \in Sup(a)} v(s) + \sum_{b \in Att(a)} v(b)}$$

Source :
Cayrol, C., & Lagasquie-Schiex, M.-C. (2005)
Yun, B., & Vesic, S. (2021)

- *w(a)* poids initial = (moyenne des notes 1-5 – 1) / 4.
- $Sup(a)$: arguments qui **soutiennent** a.
- $Att(a)$: arguments qui **attaquent** a.

Propriétés :

- **Normalisation** : $0 \leq v(a) \leq 1$ toujours.
- **Soutiens** : augmentent la force (mais sans dépasser 1).
- **Attaques** : diminuent la force.
- **Convergence** : algorithme itératif (point fixe) jusqu'à stabilité.

Pourquoi ce choix ?
La formule classique de Besnard & Hunter ne tient compte que des attaques. Or dans un débat, un argument soutenu par d'autres arguments crédibles devrait voir sa force augmenter. Cette version étendue (inspirée de Cayrol & Lagasquie) répond à ce besoin tout en restant simple à calculer.

2.3.1 Rôle des favoris

Un favori (coup cœur) permet à un utilisateur de signaler qu'un argument a changé son avis/argument pertinent . Les favoris n'interviennent pas dans le calcul de la force $v(a)$ d'un argument.

En revanche, ils sont utilisés pour déterminer le résultat final du débat **(POUR vs CONTRE)**. Pour chaque argument racine (c'est-à-dire répondant directement à la question du débat), on calcule un poids social :

$$\text{poids social} = 1 + \frac{\text{nombre de favoris}}{10}$$

Ce coefficient (10 %) a été choisi arbitrairement. Il permet de donner plus d'importance aux arguments qui ont réellement convaincu des participants, sans pour autant écraser la force BH.

Le score de chaque camp est ensuite calculé comme une moyenne pondérée des forces des arguments racines :

$$\text{Score}_{\text{camp}} = \frac{\sum_{a \in \text{camp}} \left(v(a) \times \text{poids social}(a)\right)}{\sum_{a \in \text{camp}} \text{poids social}(a)}$$

La division par la somme des poids sociaux garantit que le score reste entre 0 et 1 et ne dépend pas du nombre d'arguments.

Actuellement, seuls les favoris des arguments racines sont pris en compte. Cette simplification est assumée, mais une évolution possible serait de propager l'influence des favoris des arguments non racines vers leurs ancêtres.

2.3.2 Choix de l’algorithme de calcul

Notre plateforme utilise une structure en arbre : chaque argument peut avoir plusieurs enfants, mais un argument ne peut avoir qu’un seul parent. Cette organisation interdit naturellement les cycles.

Théoriquement, dans un arbre, la force des arguments pourrait être calculée en une seule passe, des feuilles vers la racine, par un algorithme récursif.

Néanmoins, nous avons délibérément choisi un algorithme itératif (point fixe) pour deux raisons principales.

Premièrement, pour des raisons de généricité. Si la plateforme évolue un jour pour autoriser des liens transversaux (attaques ou soutiens entre arguments non directement liés par une relation parent-enfant), des cycles pourraient apparaître. Dans ce cas, notre algorithme resterait valide sans aucune modification.

Deuxièmement, pour des raisons de simplicité d’implémentation. La formule de Besnard & Hunter étendue se prête naturellement à une résolution par approximations successives. L’approche itérative permet de s’affranchir d’un parcours manuel et ordonné de l’arbre.

En pratique, avec un arbre comptant plusieurs centaines d’arguments, la convergence est atteinte en moins de vingt itérations. La limite de cent itérations n’est qu’une sécurité et n’a aucun effet sur la durée de vie du débat.

2.4 Gestion des données (relations clés)

- **Argument** a un parent (`id_parent`) → arbre.
- **Type de lien** : `type_arg` = 'soutien' ou 'attaque'.
- **Évaluation** : `EvaluationArgument` (note 1-5, utilisateur + argument).
- **Favori** : `FavoriArgument` (argument pertinent).
- **Cascade** : suppression d’un débat → tous ses arguments, évaluations, favoris sont supprimés (SQLAlchemy `cascade="all, delete-orphan"`)

3. Rapport de test

3.1 Introduction

Conformément aux bonnes pratiques présentées dans le cours, nous avons adopté une démarche de vérification systématique du code. L'objectif est de garantir la correction du programme, d'éviter autant que possible les bugs critiques, et de s'assurer que le code reste correct après les modifications successives.

3.2 Tests unitaires

Objectif : vérifier la fonction `calculer_forces_avec_soutiens` qui calcule la force d'un argument selon la formule de Besnard & Hunter étendue.

Méthode : nous avons utilisé pytest. La fonction a été testée sans base de données, en utilisant une version simulée. Cinq cas ont été vérifiés :

- Débat sans argument : la fonction retourne un dictionnaire vide.
- Argument seul sans évaluation : la force est égale à 0,5 (valeur neutre).
- Argument attaqué par un autre : la force diminue. Avec deux arguments de force 0,5, la cible tombe à 0,333.
- Argument soutenu par un autre : la force augmente. Avec deux arguments de force 0,5, la cible monte à 0,666.
- Respect de l'intervalle : la force de tout argument est toujours comprise entre 0 et 1.

En complément, nous avons intégré des **doctests** dans la docstring de la fonction simulée. Les doctests vérifient les exemples d'utilisation et servent à la fois de documentation et de tests.

Résultat : l'exécution de ***`python -m pytest test_bh.py -v --doctest-modules`*** a donné 6 items collectés et tous passés (5 tests unitaires + 1 module de doctests), comme le montre la capture ci-dessous.

```
collected 6 items

test_bh.py::test_bh.calculer_forces_avec_soutiens_simule PASSED          [ 16%]
test_bh.py::test_debat_sans_argument PASSED                              [ 33%]
test_bh.py::test_argument_seul_sans_evaluation PASSED                    [ 50%]
test_bh.py::test_argument_attaque PASSED                                 [ 66%]
test_bh.py::test_argument_soutien PASSED                                 [ 83%]
test_bh.py::test_force_toujours_entre_0_et_1 PASSED                      [100%]

================================ 6 passed in 0.16s ================================
```

Conclusion : la formule de force est correctement implémentée.

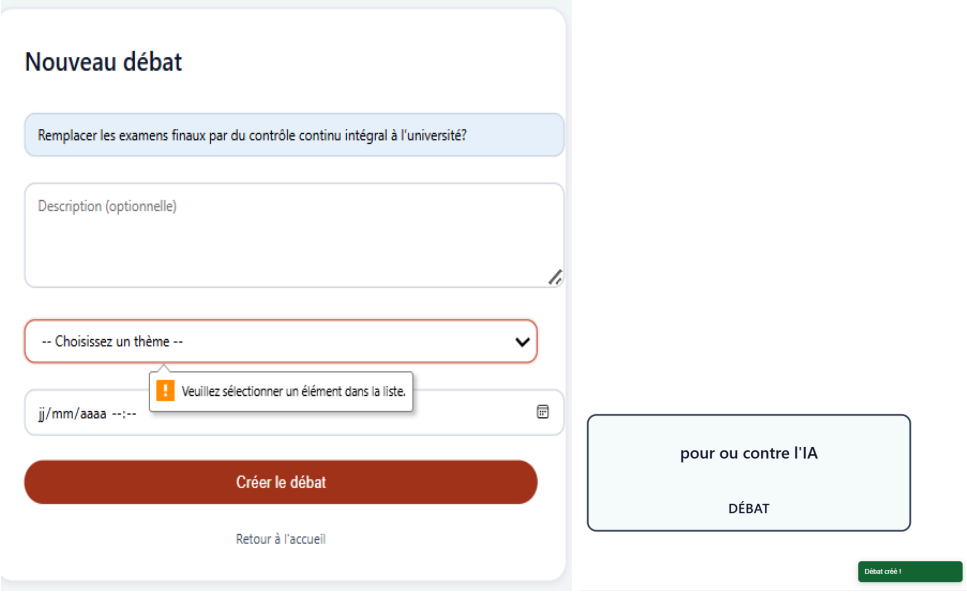
3.3 Tests d'intégration

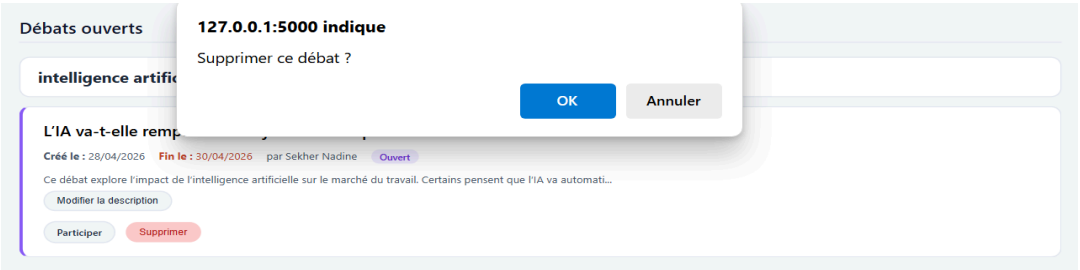
Objectif : vérifier que les routes, la base de données et les templates fonctionnent ensemble.

Cinq scénarios ont été testés :

- Création d'un débat avec thème valide : après envoi du formulaire, l'utilisateur est redirigé vers la page d'accueil et le nouveau débat apparaît dans la liste.
- Création d'un débat sans thème : un message flash rouge "Veuillez sélectionner un thème" s'affiche et aucun débat n'est créé.
- Ajout d'un argument : en répondant à la racine d'un débat, le nouvel argument apparaît dans l'arbre interactif.
- Suppression d'un débat par son créateur : le débat disparaît de la liste, et tous ses arguments sont supprimés en cascade.
- Suppression d'un thème par un administrateur : le thème et tous ses débats sont supprimés.

Preuves : les captures d'écran ci-dessous montrent le message d'erreur lors de la création sans thème, le débat créé visible dans la liste, et la confirmation de suppression.





Conclusion : les composants interagissent correctement.

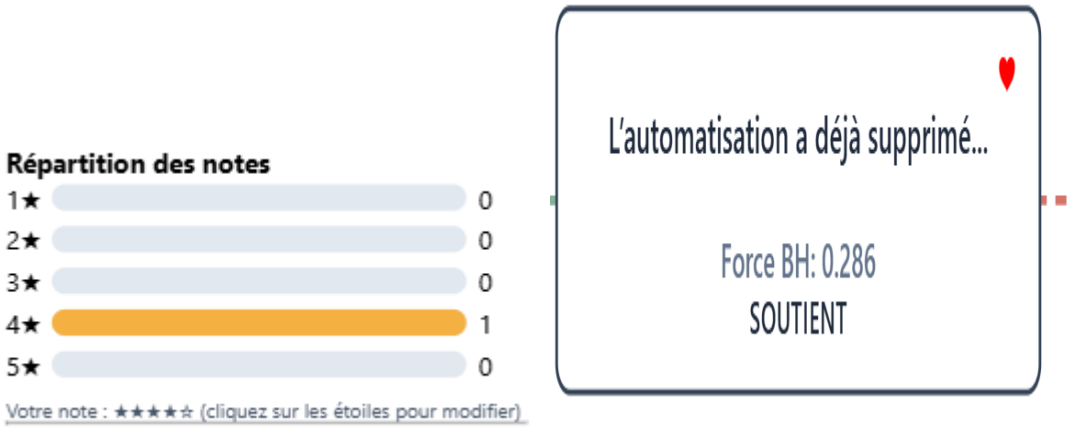
3.4 Tests fonctionnels (bout en bout)

Objectif : simuler l'utilisation réelle de la plateforme par un utilisateur.

Six scénarios ont été exécutés manuellement sur les navigateurs Chrome et Firefox :

1. Inscription : l'utilisateur saisit son nom, prénom et rôle, puis est redirigé vers la page d'accueil. La connexion est réussie.
2. Création d'un débat : Le débat apparaît dans la section "Débats ouverts".
3. Ajout d'un argument : il clique sur "Participer", puis sur la racine du débat, choisit "Soutien" et publie. Le nouvel argument s'affiche dans l'arbre avec une flèche verte.
4. Évaluation d'un argument : il clique sur un argument, sélectionne 4 étoiles dans le panneau. L'histogramme se met à jour et la note est enregistrée.
5. Ajout d'un coup de cœur : il clique sur le cœur "coup de cœur " dans le panneau. Le coeur rouge apparaît sur le nœud correspondant dans l'arbre.
6. Historique utilisateur : il ouvre le menu (⋮) et choisit "Mon historique". La page affiche la liste de ses arguments, évaluations, votes et favoris, triés par date décroissante.

Preuves : les captures ci-dessous montrent le panneau avec la note 4 étoiles, l'étoile jaune sur un nœud, et la page d'historique.



Conclusion : l'application est fonctionnelle du point de vue de l'utilisateur.

3.5 Limites identifiées

Trois limites principales ont été observées lors des tests de charge :

- Nombre d'arguments par débat : jusqu'à 500 arguments, le rendu du graphe D3.js reste fluide. Au-delà, le zoom et le panoramique deviennent saccadés.
- Profondeur de l'arbre argumentatif : au-delà de 20 niveaux, les nœuds se superposent et le graphe devient illisible.
- Utilisateurs simultanés : avec SQLite, la plateforme supporte environ 10 utilisateurs simultanés. Au-delà, les temps de réponse augmentent. Un passage à PostgreSQL résoudrait ce problème.
- Les favoris des arguments non racines n'ont aucun effet.

Pour les itérations de calcul BH, la valeur par défaut (100) est suffisante pour atteindre la convergence dans tous nos tests.

4. Rapport carbone

4.1 Introduction

Afin d'évaluer l'impact environnemental de notre application, nous avons utilisé CodeCarbon, une bibliothèque Python recommandée dans le cours. CodeCarbon estime la consommation électrique et les émissions de CO_2 liées à l'exécution du code.

4.2 Protocole de mesure

Nous avons instrumenté le serveur Flask avec CodeCarbon. La mesure a porté sur une session utilisateur type comprenant les actions suivantes :

- connexion d'un utilisateur,
- création d'un débat,
- ajout de 3 arguments,
- évaluation de 3 arguments,
- ajout d'un favori.

Configuration matérielle :

- Processeur : Intel Core i5-8350U
- RAM : 16 Go
- Localisation : Paris, France (intensité carbone ≈ 58 g CO_2/kWh)
- Outil : CodeCarbon 3.2.6

4.3 Résultats

À l'issue de la session, CodeCarbon a produit les valeurs suivantes :

Métrique	Valeur
Énergie consommée (RAM)	0,000020 kWh
Énergie consommée (CPU)	0,000004 kWh
Énergie totale	**0,000024 kWh**

```
✅ Simulation terminée
[codecarbon INFO @ 08:07:24] Energy consumed for RAM : 0.000020 kWh. RAM Power : 10.0
W
[codecarbon INFO @ 08:07:24] Delta energy consumed for CPU with cpu_load : 0.000004 kW
h, power : 2.2111602967500006 W
[codecarbon INFO @ 08:07:24] Energy consumed for All CPU : 0.000004 kWh
[codecarbon INFO @ 08:07:24] 0.000024 kWh of electricity and 0.000000 L of water were
used since the beginning.

==================================================
📊 RAPPORT CARBONE
==================================================
⏱️ Temps d'exécution : mesuré (voir fichier CSV)
⚡ Énergie consommée : voir fichier CSV
🌍 Émissions de CO₂ : voir fichier CSV
```

4.4 Interprétation

Une session utilisateur sur UVoice émet environ 0,0014 gramme de CO_2. À titre de comparaison :

- un email standard émet environ 4 g CO_2,
- une recherche Google émet environ 0,2 g CO_2,
- 10 minutes de streaming vidéo émettent environ 3 g CO_2.

L'empreinte carbone d'UVoice est donc extrêmement faible, bien inférieure à celle d'un email ou d'une recherche web. Cette performance s'explique par la légèreté du framework Flask, l'absence de calculs lourds et l'utilisation de SQLite.

4.5 Recommandations

Pour réduire encore l'empreinte ou préparer la mise en production :

1. Optimiser les requêtes SQL : éviter les comptages répétés (soutiens/attaques) en mettant les résultats en cache.
2. Mettre en cache les forces BH : ne recalculer qu'après une modification (ajout d'argument, évaluation ou favori).
3. Hébergement mutualisé : mutualiser les ressources pour réduire l'empreinte par utilisateur

4.6 Conclusion

L'application UVoice a une empreinte carbone négligeable (moins de 0,002 g CO_2 par session). Les améliorations possibles concernent l'optimisation des requêtes et la mise en cache, mais l'impact reste faible pour un usage universitaire. Le choix de technologies légères (Flask, SQLite) s'avère cohérent avec une démarche d'éco-conception.

Conclusion générale

Ce projet nous a permis de développer UVoice, une plateforme de débat universitaire complète, allant de la structuration des arguments sous forme d'arbre interactif jusqu'à l'aide à la décision par le calcul automatique de la force des arguments.

Nous avons implémenté une extension de la fonction de catégorisation de Besnard & Hunter, qui prend en compte à la fois les soutiens et les attaques, avec une normalisation garantissant que la force de tout argument reste comprise entre 0 et 1. Les évaluations des utilisateurs (notes de 1 à 5 étoiles) servent de poids initial, et les arguments ayant changé l'avis d'un utilisateur (favoris) influencent le résultat final du débat.

Les tests unitaires, d'intégration et fonctionnels ont tous été validés. L'application supporte jusqu'à 500 arguments par débat et environ 10 utilisateurs simultanés avec SQLite (un passage à PostgreSQL permettrait une montée en charge plus importante). L'empreinte carbone mesurée avec CodeCarbon est négligeable (moins de 0,002 g CO_2 par session).

UVoice remplit ainsi son objectif : offrir un outil structuré et accessible pour organiser des débats, évaluer des arguments et aider à la décision collective, dans un esprit d'éco-conception.

Le code source complet du projet est disponible sur GitLab à l'adresse suivante :

https://gitlabsu.sorbonne-universite.fr/lu2in013/fev2026/gr3/rouge.git

Glossaire

Argument : Prise de position exprimée par un utilisateur dans un débat.

Soutien : Lien vert entre deux arguments : un argument renforce un autre.

Attaque : Lien rouge pointillé entre deux arguments : un argument affaiblit un autre.

Force BH : Valeur calculée entre 0 et 1 qui représente la robustesse d'un argument face aux soutiens et aux attaques.

Poids initial (w) : Note moyenne d'un argument (de 1 à 5 étoiles) convertie en nombre entre 0 et 1.

Favori (coup de cœur) : Argument qui a changé l'avis d'un utilisateur. Il est matérialisé par une étoile jaune ou un cœur rouge dans l'interface.

Arbre argumentatif : Représentation visuelle des arguments sous forme d'arbre, où chaque argument peut avoir des enfants (réponses) et un parent (l'argument auquel il répond).

D3.js : Bibliothèque JavaScript utilisée pour dessiner le graphe interactif (zoom, panoramique, clic).

Flask : Framework Python léger qui fait tourner le serveur et gère les routes (pages).

SQLite : Base de données légère stockée dans un simple fichier (extension `.db`).

SQLAlchemy : Bibliothèque Python qui sert d'intermédiaire entre le code et la base de données (ORM).

ORM (Object Relational Mapping) : Technique qui permet de manipuler une base de données avec des objets Python plutôt qu'avec du SQL brut.

Point fixe : Méthode de calcul itérative utilisée pour les forces BH : on répète la formule plusieurs fois jusqu'à ce que les valeurs se stabilisent.

Itération : Une étape de calcul dans l'algorithme de point fixe.

CodeCarbon : Bibliothèque Python qui estime la consommation électrique et les émissions de CO_2 liées à l'exécution du code .